

SageMath: Computing with the Queen of Mathematics

In the previous article in this series (published in the January 2026 issue of *OSFY*), we concluded our discussion of cybersecurity and the implementation of various cybersecurity algorithms using SageMath.

We will now explore SageMath libraries and packages that can be applied to the field of mathematics known as number theory.



Number theory has wide-ranging applications in computer science, including cybersecurity, algorithm design, random number generation, error-correcting codes, computer graphics, machine learning, artificial intelligence, and blockchain. The following quote by the famous mathematician Carl Friedrich Gauss — often called the ‘Prince of Mathematics’ — highlights the importance of number theory: “Mathematics is the queen of the sciences, and number theory is the queen of mathematics.” (In Gauss’s time, number theory was commonly referred to as arithmetic.)

Let us begin by discussing a fundamental requirement in computation: the ability to store very large integers without overflow. As we already know, Python can handle arbitrarily large integers, and SageMath inherits this capability. Let us look at a few examples that demonstrate how SageMath works with extremely large numbers. As in our previous articles involving heavy mathematical computations, we will use CoCalc rather than running the code locally on our systems. Consider the SageMath code shown below.

```
n = 10^1000
print(n)
```

In the above code the integer 10^{1000} — which, by definition, has no fractional part — is stored in the variable n . The caret symbol (^) denotes exponentiation, and SageMath stores the resulting value exactly, without any loss of precision. However, I will not display the full number here, as it would occupy at least one paragraph. Let us instead turn to even larger numbers, expressed using notations such as Knuth’s up-arrow notation ($\uparrow$), introduced by the famous computer scientist Donald Knuth.

Let us now understand Knuth’s up-arrow notation. Exponentiation is written as $a \uparrow b$, which corresponds to a^b in SageMath. For example, $3 \uparrow 4 = 3^4 = 81$. Now consider the double-arrow operator ($\uparrow\uparrow$). The expression $3 \uparrow\uparrow 3$ represents a power tower: $3^{(3^3)} = 3^{27} = 7625597484987$. From this, one can imagine how a number such as $3 \uparrow\uparrow 10$ would behave — it is a tower of ten 3’s stacked in exponent form. Mathematics contains notations that grow even faster, such as Graham’s number, which describes quantities far beyond everyday intuition. To work with such astronomically large expressions, SageMath relies on a technique called lazy evaluation. Lazy evaluation means an expression can be stored symbolically without being fully computed until its